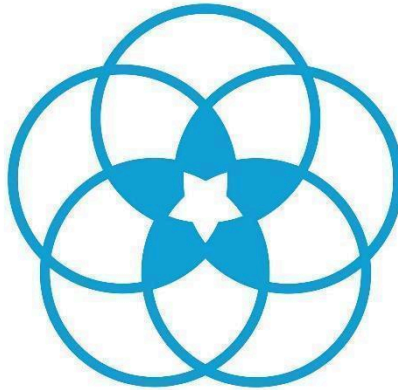




Health Information System Program Middle East and North Africa (HISP MENA)

Solution Design

Document No.: SD-202503-1

Version: 1.8

Confidentiality Notice: This document and the information contained within are strictly confidential and intended solely for the use of the designated recipient(s). Unauthorized use, distribution, or duplication of this document or any part of its content is prohibited and may be subject to legal action. If you are not the intended recipient, please notify the sender immediately by email and delete this document from your system.

HISP MENA

Date: Sept 01, 2025

Table 1: Version control

Versio n	Date	Description of Changes
1.0	2025-03-05	Initial creation of the document.
1.2	2025-04-05	Logging module design
1.3	2025-05-05	Routes design update
1.4	2025-06-05	Scope update and user management
1.5	2025-07-05	Integration and error mapping
1.6	2025-09-01	Final review for the draft version
1.7	2025-10-20	Document review completed, tables and parameters verified, and minor corrections applied
1.8	2025-10-27	Final alignment of architecture details, consistency checks, and preparation for official release

Table of Contents

Overview..... 6

 Key Functionalities..... 6

High-Level Design..... 6

 Frontend Layer..... 7

 Middleware Components..... 7

 Database Layer..... 7

 Integration Layer..... 7

Low-Level Design..... 8

 System Architecture..... 8

 Database Schema (PostgreSQL)..... 9

 API Orchestration with Apache Camel..... 10

 Web Interface (jQuery & HTML)..... 10

LLD- Logging..... 11

 Table: middleware_api_call_log..... 11

 Table: audit_logs..... 13

 Logging Process Flow..... 14

 Transaction Initiation..... 14

 Processing & API Invocation..... 14

 Response Handling..... 15

 Error Handling & Reprocessing..... 15

 Integration with Middleware Components..... 15

Spring Boot (Backend).....	15
Apache Camel (API Orchestration).....	15
PostgreSQL (Database).....	15
Web-Based UI (jQuery & HTML5).....	15
Log Management & Monitoring.....	16
Security & Data Consistency.....	16
LLD - Error Mapping Configuration.....	16
Table: source_systems.....	16
Table: error_categories.....	17
Table: error_mapping.....	18
Error Mapping Process Flow.....	19
Error Detection.....	19
Lookup in Error Mapping Table.....	20
Error Logging & Response Handling.....	20
LLD - Job Scheduling and Execution Tracking.....	20
Table: job_schedules.....	20
Table: job_executions.....	21
Table: job_execution_logs.....	22
Job Scheduling Process Flow.....	23
Job Registration.....	23
Job Execution.....	24
Job Processing.....	24
Monitoring.....	24
LLD - Notifications, Templates, and Configurations.....	25
Notification Process Flow.....	29
Trigger Notification.....	29
Format Message.....	30
Send Notification.....	30
LLD - Dynamic Routes.....	30
Database Tables.....	30
Table: dynamic_routes.....	30
Dynamic Routes Process Flow.....	32
Create Initial Route.....	32
Update or Manage Route Versions.....	32
Record User Actions in Audit Table.....	32
Route Execution Logging.....	32
Auditing, Maintenance, and Version Control.....	32
LLD – Rate Limiting.....	33
Rate Limiting Process Flow.....	34
Define Global Rate Limits.....	34

Define Camel Integration Limits.....	34
Configure Route-Specific Limits.....	34
Enforce Rate Limits at Runtime.....	34
Auditing.....	35
LLD -Integration Mapping.....	35
Integration Mapping Process Flow.....	38
Ingest / identify context.....	38
Lookup active mappings for the route.....	39
Group mappings by external API.....	39
Fetch metadata (org units) per integrated system.....	39
Execute analytics calls per IntegratedApi.....	39
Extract values from analytics responses per mapping.....	40
Build middleware response.....	40
Auditing.....	41
Transaction logging.....	41
LLD -User Management.....	42
Users & Roles Management Process Flow.....	45
User Creation & Role Assignment.....	45
Role Definition & Permission Mapping.....	46
Authentication.....	46
Authorization.....	46
Auditing & Dynamic Updates.....	46
LLD -Config Settings and Integrated Systems.....	47
Config and Integration Management Process Flow.....	50
Manage Configuration Parameters.....	50
Add or Update Integrated Systems.....	50
Apply and Use Configurations.....	50
Auditing and Traceability.....	50
Swagger API Documentation Integration.....	50
Camel Route Designer.....	51
Designer's feature comparisons.....	52
1. Kaoto.....	52
2. Camel Karavan.....	52
3. Camel Designer (VS Code Plugin).....	53
4. Fuse Tools (JBoss Tools – Eclipse).....	53
5. Hawtio.....	54
6. Synthesis / Fuse Online.....	54
Conclusion.....	56
Kaoto Web App and Kaoto VS Code Plugin.....	56
Kaoto Web App.....	56

VS Code Plugin.....	56
Integration Project Phases.....	58

Overview

The middleware will facilitate API orchestration using Apache Camel, built with Spring Boot for backend processing and jQuery & HTML for the web-based interface. PostgreSQL will be used for logging, configurations, and tracking system events.

Key Functionalities

1. **Dashboard for Monitoring:** Web-based dashboard to track system performance, API calls, and errors.
2. **Job Scheduling & Execution Tracking:** Configurable scheduler for batch or time-based processing.
3. **Error Handling:** Centralized mechanism for logging failures.
4. **Detailed Logging & Traceability:** Database-backed logging for all middleware transactions.
5. **User Management:** Role-based access control and authentication.
6. **Data Mapping:** Standardized transformation logic for incoming and outgoing data.
7. **Avoiding Point-to-Point Integration Issues:** Implementing best practices like API Gateway and Event-Driven patterns.
8. **Data Consistency & Quality:** Validations, deduplication, and integrity checks.
9. **Web-Based Administration Interface:** UI for monitoring, configurations, and control.

High-Level Design

The Middleware System has been designed as a central integration hub to streamline data exchange between the Ministry of Health (MoH) systems and external healthcare platforms. It ensures secure, reliable, and standardized communication while reducing complexity and duplication across systems.

At a high level, the architecture is composed of the following layers:

Frontend Layer

Provides a web-based interface for administrators and authorized users. It includes modules for:

1. User Management: managing roles, permissions, and secure access.
2. Transaction Management: monitoring and controlling data transactions.
3. Configuration Management: managing system parameters, integration settings, and runtime configurations.

Middleware Components

Core functional modules that handle system operations, including:

1. Transaction Processing: API calls and manages request/response cycles.
2. Logging & Error Handling: captures detailed logs and standardizes error management.
3. Scheduler: enables automated and time-based workflows.
4. Data Mapping: ensures consistent transformation of data across integrated systems.
5. Dashboard & Logs Extraction: provides real-time monitoring and reporting capabilities.

Database Layer

Centralized repository for storing configurations, transaction logs, audit trails, and system metadata. The database supports indexing and search functions for efficient retrieval and compliance.

Integration Layer

Middleware provides APIs to external systems. In this phase, the focus is on:

1. Hakeem Integration: enabling secure exchange of clinical data from electronic medical records into the MoH environment.
2. HealthMap Integration: delivering aggregated health data to support visualization, analytics, and decision-making.

This modular architecture ensures flexibility, scalability, and maintainability, allowing new integrations to be added seamlessly while maintaining consistent security and governance standards.

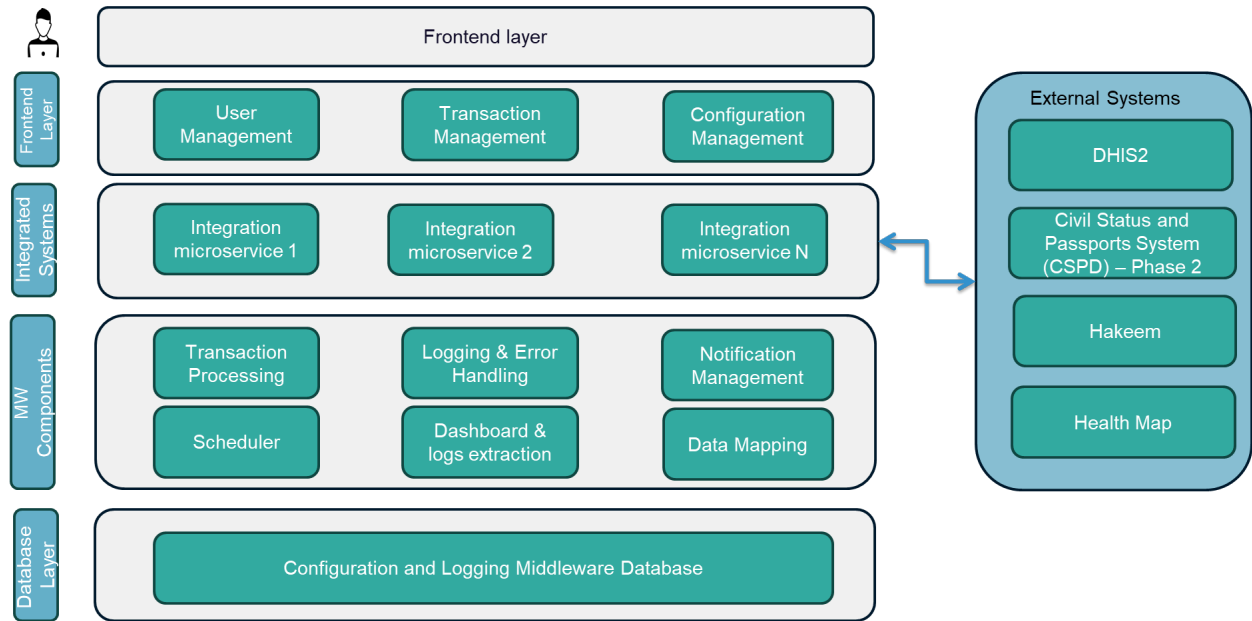


Figure 1: High-Level Design

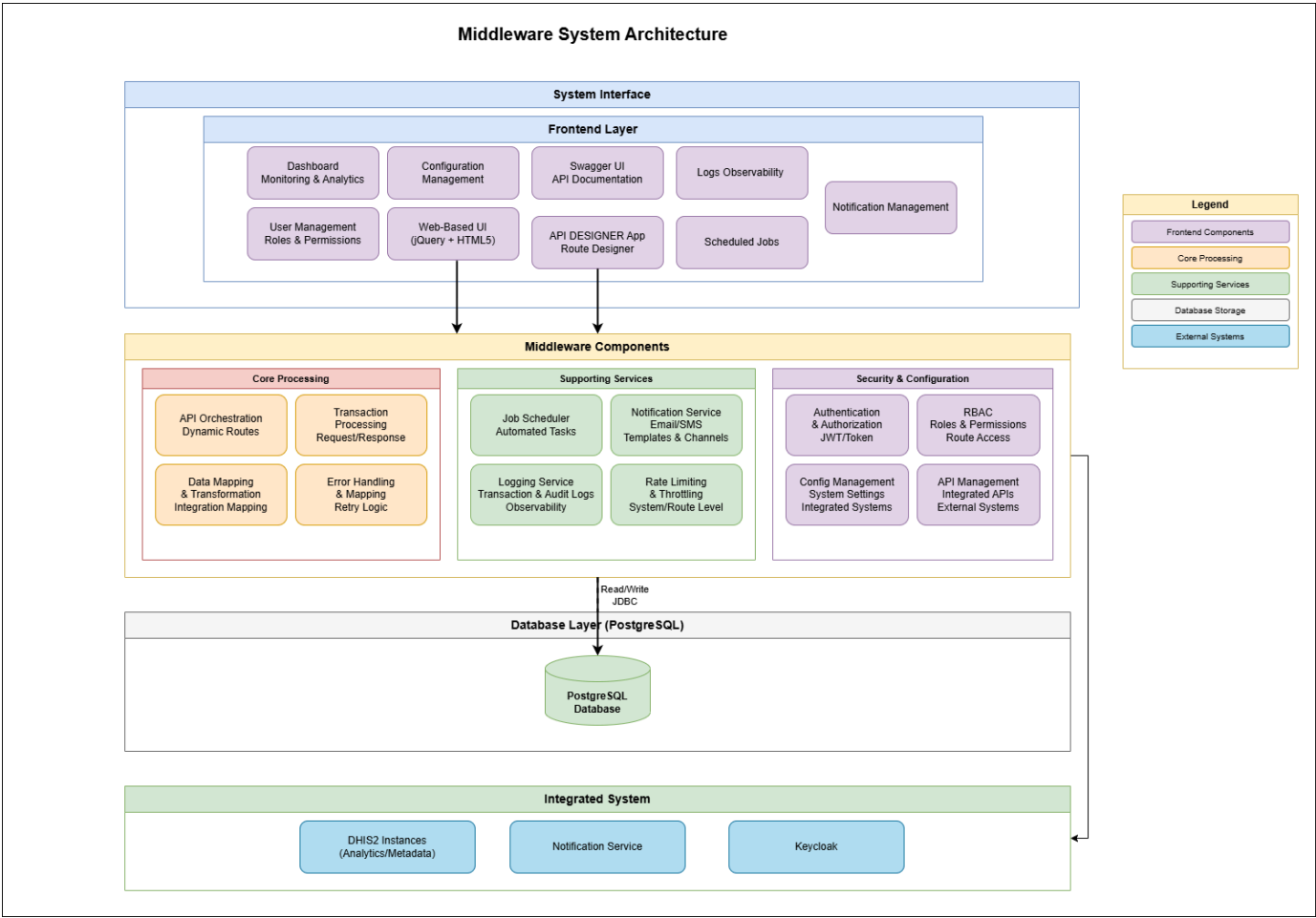
Low-Level Design

System Architecture

1. Spring Boot (Backend Service)
 - Manages API processing, business logic, and database interaction.
2. Apache Camel (Integration Layer)
 - Routes, transforms, and orchestrates APIs.
3. PostgreSQL (Database)
 - Stores logs, configurations, and transactional data.
4. jQuery & HTML (Frontend)

System Architecture Overview

The Middleware Platform is designed as a centralized integration hub enabling secure, scalable, and configurable data exchange between MoH systems and external healthcare platforms. The architecture follows a layered design, combining a web-based administration interface, a modular middleware engine for routing, transformation, security, and automation, and a PostgreSQL data layer for configurations, audit logs, and system metadata. It supports seamless interoperability with external systems such as DHIS2, Hakeem, and Keycloak, ensuring standardized communication, strong access control, and real-time monitoring across all integrated services.



Database Schema (PostgreSQL)

This is the list of database tables that the Middleware system creates upon start.

Table Name	Description
users	Stores user information such as username, email, password, and roles.
role	Defines user and System roles in the system (e.g., ADMIN, VIEWER, HAKEEM), including role names and descriptions.
users_roles	Maps users to roles in a many-to-many relationship, defining which users have which roles.
permissions	Stores the system-level authorities (permissions) assigned to real (human) users, such as logs:view , route:create , and similar privileges.
role_permissions	Maps multiple system-level authorities (permissions) to real user roles in a many-to-many relationship, defining which permissions are granted to each real user role.
routes_permissions	Stores the route-level permissions (route IDs) that define which routes system user roles can access.
role_routes_permissions	Maps multiple route-level permissions to system user roles in a many-to-many relationship, defining which routes each system role can access.
user_tokens	Stores authentication and access tokens for both real (human) users and system users, including the token ID, creation timestamp, expiration time, validity status, and the token string itself.
dynamic_routes	Contains defined routes that should be exposed as API endpoint
dynamic_route_audit	Logs all user actions on the Dynamic Routes/APIs page, including creating, editing, pausing, resuming, and reverting versions of routes.
error_mapping	Maps raw error messages from external APIs to standardized error codes and messages used by the system.
error_categories	defines categories for grouping and classifying errors across middleware integrations.
source_systems	stores external or internal systems that interact with the middleware
audit_logs	Logs all user and system actions for auditing purposes, capturing the action, user, and context.
middleware_api_call_log	Logs details of each inbound and outbound API call made, including request/response information and performance metrics.
config	A generic key-value store for storing configuration settings used by the system, such as general APIs limits or feature flags.
job_schedules	Defines scheduled jobs, which trigger workflows or API calls based on time-based schedules or events.
job_execution_logs	Logs the execution of scheduled jobs, tracking status, errors, retry attempts, and execution times.
job_executions	Logs all actions performed by real users on the Scheduled Jobs page, such as creating, pausing, resuming, or editing jobs.
notification_channels	Stores channel configurations (e.g., email, SMS) including unique code, type, JSON config/credentials, active flag and audit fields.

notification_action_log	Audit log of administrative actions and notification-related events (action, details, admin email, event timestamp).
notification_groups	Named recipient groups (code, description) that aggregate multiple receivers for bulk sends; includes audit fields.
notification_receivers	Individual recipients (name, email, phone) and their audit metadata; referenced by groups.
notification_groups_receivers	Maps receivers to their respective notification groups.
notification_templates	Reusable templates (unique code, name, channel type, subject, body) used to render messages; includes audit fields.
notification_logs	Runtime delivery records: which group/template/channel was used, delivery status (PENDING/SUCCESS/FAILED), errorMessage, user info, timestamps.
rate_limit_config	Stores the request limits for all internal system APIs, including the maximum allowed number of requests within a specific time window (e.g., 200 requests per hour).
camel_rate_limit	Stores the request limits for all dynamic routes or APIs starting with /camel, including the maximum allowed requests within a specific time window (e.g., 200 requests per hour).
custom_camel_rate_limit	Stores request limits for specific dynamic routes or APIs starting with /camel, including the maximum allowed requests within a specific time window (e.g., 200 requests per hour) and the associated route ID.
integrated_system	Stores settings for external systems, including connection details, credentials, and identifying information
integrated_apis	Stores all API definitions that are available to be used inside integration mappings including their endpoint, method, and configuration details. It serves as the source list of APIs that can be linked or mapped together in the integration layer.
integration_mapping	Defines interactions between APIs from integrated_apis by specifying field-to-field mappings and transformation rules. Controls how data flows and is converted between connected systems during integration.

API Orchestration with Apache Camel

Apache camel Routes for Processing will be implemented to modularize the integration functionality implementation.

Web Interface (jQuery & HTML)

1. Dashboard: Live status of transactions, errors, and performance.
2. Job Scheduler UI: Configure, enable, disable, and monitor scheduled jobs.
3. Error Logs & Alerts: View failed transactions and retry processing.
4. User Management: Manage roles and permissions.

LLD- Logging

The middleware requires a detailed transaction logging mechanism to ensure traceability, debugging, and compliance with integration requirements. This logging system will capture transaction details, errors, and processing stages to allow monitoring and troubleshooting.

The transaction logs will be stored in PostgreSQL, maintaining structured records for traceability.

Table: middleware_api_call_log

Logs inbound/outbound API calls.

Column Name	Type	Description
id	BIGINT (PK)	Unique log entry ID
transaction_id	VARCHAR(36) UNIQUE NOT NULL	Unique transaction identifier for this API call
route_id	VARCHAR(255) NOT NULL	Identifier of the route invoked
api_endpoint	VARCHAR(255)	API endpoint called
request_method	VARCHAR(10)	HTTP method used (GET, POST, etc.)
request_url	VARCHAR(2048)	Full request URL
request_path	VARCHAR(1024)	Request path

request_query	VARCHAR(2048)	Query parameters
request_headers	TEXT	HTTP request headers
request_body	TEXT	HTTP request body
status	VARCHAR(20)	Call status (SUCCESS/FAILURE)
response_code	INT	HTTP response code
response_headers	TEXT	HTTP response headers
response_body	TEXT	HTTP response body
received_at	TIMESTAMP	Timestamp when request was received
completed_at	TIMESTAMP	Timestamp when response was completed
duration_ms	BIGINT	Request processing time in milliseconds
client_ip	VARCHAR(100)	IP of the client making the request
api_key_id	BIGINT	Associated API key ID (if applicable)

user_id	VARCHAR(255)	User ID initiating the request
error_message	TEXT	Error message, if any
retry_count	INT	Number of retries performed
attempt_no	INT NOT NULL	Attempt number for this transaction
source_transaction_uuid	VARCHAR(36) NOT NULL	UUID of the source transaction (for correlation)

Table: audit_logs

Tracks user/system actions.

Column Name	Type	Description
id	BIGINT (PK)	Unique audit log ID
user_name	VARCHAR(255)	Name of the user performing the action
method	VARCHAR(50)	HTTP method of the request (GET, POST, etc.)
api_path	VARCHAR(1024)	API path accessed
query_string	VARCHAR(2048)	Query string of the request

response_status	INT	HTTP response status code
start_time	TIMESTAMP	Timestamp when request processing started
end_time	TIMESTAMP	Timestamp when request processing ended
duration_ms	BIGINT	Total processing time in milliseconds
request_headers	TEXT	Full HTTP request headers
request_body	TEXT	HTTP request body
response_headers	TEXT	Full HTTP response headers
response_body	TEXT	HTTP response body

Logging Process Flow

Transaction Initiation

1. When a new request enters the middleware, a log entry is created with the status "**IN-PROGRESS**".
2. The request payload is stored for debugging purposes.
3. The log includes metadata such as Transaction ID, Source System,, and API Endpoint.

Processing & API Invocation

1. Middleware processes the request and routes it to the respective Route

Response Handling

1. When a route is executed (successfully or with an error), the log entry status is updated to "Completed".
2. The corresponding response data and response code are recorded, indicating whether the transaction succeeded or failed.
3. A transaction remains in "In Progress" status only while it is being processed.
4. Both external system latency and total latency are stored in the database for monitoring and analysis.

Error Handling & Reprocessing

1. Failed transactions are displayed in the admin interface with retry options.
2. Users can trigger a manual retry, updating the log with a new attempt entry.
3. Each retry attempt is recorded to track resolution attempts.

Integration with Middleware Components

Spring Boot (Backend)

1. Handles log creation, updates, and retrieval.
2. Provides API endpoints for log access and searching.

Apache Camel (API Orchestration)

1. Automatically captures logs at various stages of transaction processing.
2. Ensures transaction tracking across integrations.

PostgreSQL (Database)

1. Stores logs persistently.
2. Allows indexing for efficient search and filtering.

Web-Based UI (jQuery & HTML5)

1. Displays transaction logs in a dashboard with filters.
2. Allows users to view details, search logs, and retry failed transactions.

Log Management & Monitoring

- 1. Logs are stored permanently for compliance but can be archived periodically.
- 2. Users can filter logs by Transaction ID, Status, Date Range, or System.

Security & Data Consistency

- 1. Access Control: Only authorized users can view or export logs.
- 2. Data Integrity: Ensuring logs are write-once and immutable to prevent tampering.
- 3. Data Integrity: Ensure all integration processing steps are completed successfully. And rolling back the complete steps in case of failure.
- 4. Performance Optimization: Indexed queries ensure efficient retrieval of logs.

LLD - Error Mapping Configuration

The Error Mapping Configuration module standardizes errors handling across middleware integrations by mapping system-generated errors to predefined codes and messages. This ensures consistent error reporting, improved troubleshooting, and user-friendly messages.

The error mapping configuration is stored in a database table, allowing dynamic updates without changing application logic.

Table: source_systems

Defines a Source System.

Column Name	Type	Description
id	BIGINT (PK)	Unique source system ID
name	VARCHAR(100) NOT NULL UNIQUE	Name of the source system
description	TEXT	Description of the source system

active	BOOLEAN NOT NULL	Whether the source system is active
created_at	TIMESTAMP NOT NULL	Timestamp when the record was created
updated_at	TIMESTAMP NOT NULL	Timestamp when the record was last updated
created_by	VARCHAR(255) NOT NULL	User who created the record.
updated_by	VARCHAR(255) NOT NULL	User who last updated the record.

Table: error_categories

Defines an Error Category.

Column Name	Type	Description
id	BIGINT (PK)	Unique error category ID
name	VARCHAR(100) NOT NULL UNIQUE	Name of the error category
description	TEXT	Description of the error category

active	BOOLEAN NOT NULL	Whether the error category is active
created_at	TIMESTAMP NOT NULL	Timestamp when the record was created
updated_at	TIMESTAMP NOT NULL	Timestamp when the record was last updated
created_by	VARCHAR(255) NOT NULL	User who created the record.
updated_by	VARCHAR(255) NOT NULL	User who last updated the record.

Table: error_mapping

Defines an Error Mapping.

Column Name	Type	Description
id	BIGINT (PK)	Unique mapping ID
code	VARCHAR(255) UNIQUE NOT NULL	Unique Mapping code identifier
route_id	VARCHAR(255) NOT NULL	Reference to the dynamic route ID
route_path	VARCHAR(500)	API path from the route (e.g., /api/users)
source_system_id	BIGINT (FK → source_systems.id)	Optional reference to the source system

raw_error_substring	TEXT NOT NULL	Substring to match in the raw error
match_type	VARCHAR(20) NOT NULL	Matching type: CONTAINS, EQUALS, REGEX
mapped_error_code	VARCHAR(50) NOT NULL	Standardized error code
mapped_message	TEXT NOT NULL	User-friendly error message
error_category_id	BIGINT (FK → error_categories.id)	Optional reference to an error category
http_status_code	INT NOT NULL	HTTP status code to return
language	VARCHAR(10) DEFAULT 'en'	Message language
active	BOOLEAN NOT NULL	Whether the mapping is active
created_by	BIGINT NOT NULL	ID of the user who created the mapping
updated_by	BIGINT	ID of the last user who updated the mapping
created_at	TIMESTAMP NOT NULL	Timestamp when the mapping was created
updated_at	TIMESTAMP NOT NULL	Timestamp when the mapping was last updated

Error Mapping Process Flow

Error Detection

1. Middleware detects an error during API processing.
2. Captures the error code and message from the external system response.

3. The API designer can call predefined component to retrieve the corresponding error mapping

Lookup in Error Mapping Table

- 1- Middleware queries the Error Mapping Table using:
 - a. Source System
 - b. External Error Code or if external error message is included in the complete raw error message from external system.
- 2- If a match is found, the corresponding Error Code and Mapped Message is retrieved.

Error Logging & Response Handling

1. The standardized error message is:
 - a. Logged in the Transaction Logs Table.
 - b. Sent as a response to the client.
 - c. Triggered for notifications (if applicable).

LLD - Job Scheduling and Execution Tracking

The Job Scheduling and Execution Tracking module automates the execution of periodic and event-driven tasks within the middleware. It ensures reliable job execution, tracks job statuses, and provides visibility into scheduled and completed jobs.

Table: job_schedules

Defines Scheduled Jobs.

Column Name	Type	Description
id	BIGSERIAL / BIGINT	Primary key. Auto-generated unique identifier for each job schedule.
job_name	VARCHAR	Name of the scheduled job.
description	VARCHAR	Description of the job's purpose or behavior.

schedule_expression	VARCHAR	Expression (e.g., CRON format) defining when the job should run.
api_endpoint	VARCHAR (NOT NULL)	Target API endpoint to be triggered when the job executes.
method	VARCHAR	HTTP method (e.g., GET, POST, PUT) used to call the API endpoint.
headers	VARCHAR(1000)	HTTP headers (JSON or stringified key-value pairs) for the API request.
payload	VARCHAR(1000)	Request body/payload to send with the API call.
enabled	BOOLEAN	Indicates whether this schedule is enabled (true) or disabled (false).
last_execution_time	TIMESTAMP	The timestamp when the job was last executed.
created_by	BIGINT (NOT NULL)	ID of the user/system that created the job schedule.
created_at	TIMESTAMP (NOT NULL)	The timestamp when the job schedule was created.
updated_by	BIGINT (NOT NULL)	ID of the user/system that last updated the job schedule.
updated_at	TIMESTAMP (NOT NULL)	The timestamp when the job schedule was last updated.
active	BOOLEAN	Logical flag to mark if the job schedule is currently active. Default = true
token	VARCHAR(1000)	Optional authentication or access token used when calling the target API endpoint.

Table: job_executions

Tracks User Actions Logs on Scheduled Jobs.

Column Name	Type	Description
-------------	------	-------------

id	BIGSERIAL / BIGINT	Primary key. Auto-generated unique identifier for each execution record.
scheduled_job_name	VARCHAR (NOT NULL)	Name of the scheduled job associated with this execution.
scheduled_job_method	VARCHAR (NOT NULL)	HTTP method (e.g., GET, POST, PUT) of the scheduled job.
scheduled_job_path	VARCHAR (NOT NULL)	API path/endpoint of the scheduled job.
action	VARCHAR	The action performed during execution (if applicable).
status	ENUM('SUCCESS','FAILURE','RETRY')	Execution status from the Status enum.
error_message	VARCHAR	Error details if the execution failed.
created_at	TIMESTAMP (NOT NULL)	The timestamp when this execution was recorded.
user_email	VARCHAR	Email of the user who triggered or is associated with this execution.

Table: job_execution_logs

Tracks Job Execution Logs.

Column Name	Type	Description
-------------	------	-------------

id	BIGSERIAL / BIGINT	Primary key. Auto-generated unique identifier for each execution log.
job_id	BIGINT (FK)	Foreign key referencing ScheduledJobs.id . Links log entry to a scheduled job.
start_time	TIMESTAMP	Start time of the job execution.
end_time	TIMESTAMP	End time of the job execution.
duration_ms	BIGINT	Total duration of the job execution in milliseconds.
request_headers	TEXT (LOB)	HTTP request headers used when executing the job.
request_body	TEXT (LOB)	HTTP request body payload.
response_headers	TEXT (LOB)	HTTP response headers from the API call.
response_body	TEXT (LOB)	HTTP response body returned from the API call.
response_status	INT	HTTP response status code (e.g., 200, 400, 500).
status	ENUM('SUCCESS','FAILURE','RETRY')	Execution result status from the Status enum.
error_message	VARCHAR(1000)	Error details if execution failed.

Job Scheduling Process Flow

Job Registration

1. A new job is registered via the authorized user UI with:

- a. Name, schedule (Cron), job type, and status.
- b. The job is stored in the `job_schedules` table.

Job Execution

1. Spring Boot Scheduler (Quartz) picks up the job based on its schedule.
 - a. A new entry is created in the `job_executions` table, and after execution, its status is updated to "SUCCESS" if completed successfully or "FAIL" if an error occurs.

Job Processing

1. The middleware executes the task.
2. If successful, the execution status updates to "SUCCESS".
3. If it fails, the error message is logged.

Monitoring

1. All user operations on scheduled jobs (create, update, enable/disable, manual trigger) are saved in the `job_executions` table.
2. All scheduled job runs are recorded in the `job_execution_logs` table, including start/end time, duration, response status, and errors.

LLD - Notifications, Templates, and Configurations

The Alert Notifications module enables real-time error reporting, status updates, and alerts for system events. This ensures timely responses to failures, warnings, and critical events, allowing administrators to take necessary actions.

It manages communication to groups of receivers via channels such as **Email** and **SMS**.

It is designed to:

- Allow **dynamic configuration** of notification channels (API credentials, SMTP, SMS gateway).
- Support **notification templates** with reusable body placeholders.
- Organize recipients into **groups** for bulk notifications.
- Log all notification attempts, statuses, and errors for **auditing and monitoring**.
- Track **actions** (e.g., configuration changes, group updates).

This ensures consistency, scalability, and traceability of messaging across middleware integrations.

3. Database Tables

Table: notification_templates

Defines Notification Templates.

Column Name	Type	Description
id	BIGINT (PK)	Template ID
code	VARCHAR (Unique)	Unique template code
name	VARCHAR	Template name
type	ENUM(EMAIL, SMS)	Notification type

active	Boolean	Logical flag to mark if the group is currently active. Default = true
subject	VARCHAR	Notification subject (for email)
body	TEXT	Template body (supports placeholders)
created_by	VARCHAR	User who created the template
created_at	TIMESTAMP	Creation timestamp
updated_by	VARCHAR	Last updater
updated_at	TIMESTAMP	Last update timestamp

Table: notification_receivers

Defines Receivers.

Column Name	Type	Description
id	BIGINT (PK)	Receiver ID
name	VARCHAR	Receiver name
email	VARCHAR	Email address
phone	VARCHAR	Phone number
created_by	VARCHAR	User who created the receiver
created_at	TIMESTAMP	Creation timestamp
updated_by	VARCHAR	Last updater
updated_at	TIMESTAMP	Last update timestamp

Table: notification_groups

Defines Groups.

Column Name	Type	Description
id	BIGINT (PK)	Unique group ID
name	VARCHAR	Group name
code	VARCHAR (Unique)	Unique group code

active	Boolean	Logical flag to mark if the group is currently active. Default = true
description	VARCHAR	Group description
created_by	VARCHAR	User who created the group
created_at	TIMESTAMP	Creation timestamp
updated_by	VARCHAR	Last updater
updated_at	TIMESTAMP	Last update timestamp

Table: notification_groups_receivers

Defines Group-Receiver Mapping.

Column Name	Type	Description
notification_group_id	BIGINT	Identifier of the notification group.
receivers_id	BIGINT	Identifier of the receiver assigned to the group.

Table: notification_channels

Defines Notification Channels.

Column Name	Type	Description
id	BIGINT (PK)	Unique channel ID
name	VARCHAR	Channel name (e.g., "Default Email")
code	VARCHAR (Unique)	Unique channel code
type	ENUM(EMAIL, SMS)	Channel type
config	TEXT	JSON with API credentials, SMTP/SMS config
active	BOOLEAN	Whether the channel is active
created_by	VARCHAR	User who created this channel
created_at	TIMESTAMP	Creation timestamp
updated_by	VARCHAR	Last updater
updated_at	TIMESTAMP	Last update timestamp

Table: notification_action_log

Defines actions performed by a user on Notifications.

Column Name	Type	Description
id	BIGINT (PK)	Log entry ID
action	VARCHAR	Action performed (e.g., "CREATE_GROUP")
details	VARCHAR	Details about the action
email	VARCHAR	Admin user email
eventTime	TIMESTAMP	Time of action

Table: notification_logs

Tracks sent Notifications Logs.

Column Name	Type	Description
id	BIGINT (PK)	Log ID
group	VARCHAR	Notification group used
template	VARCHAR	Template applied
channel	VARCHAR	Channel used
status	VARCHAR	PENDING / SUCCESS / FAILED
errorMessage	TEXT	Error details (if failed)
user_name	VARCHAR	User who triggered the notification
request_body	TEXT	The body content sent with the notification request.
request_header	TEXT	The headers sent with the notification request.
response_body	TEXT	The body content received in the response from the notification service.
response_header	TEXT	Headers received in the response from the notification service.
response_code	TEXT	HTTP or service response code for the notification request.
created_at	TIMESTAMP	Log creation time
sent_at	TIMESTAMP	Time notification was sent

Example Alert Templates

- **Greetings Notification Template**

```
{
  "subject": "New User",
  "body": "Dear {{name}}, thanks for joining us! We will be contacting you at {{email}} and {{phone}}."
}
```

Notification Process Flow

Trigger Notification

1. Middleware receives an event (e.g., error, workflow result, API call).
2. The user selects the **template**, **notification group** and **Channel**.

Format Message

- Template placeholders are populated with dynamic values.
- The subject and body are constructed.

Send Notification

- The user chooses the channel configuration (SMTP API, SMS gateway).
- Response is recorded with **status = SUCCESS/FAILED**.

Step 6: Logging & Auditing

- Logs are inserted into notification_logs.
- Any **changes** to channels, groups, or templates are tracked in notification_action_log.

LLD - Dynamic Routes

The **Dynamic Routes** module allows the middleware to manage API routes dynamically at runtime.

Routes are versioned, can be activated/deactivated, and are stored along with their YAML definitions.

This enables flexible updates, multiple route versions, and auditability of all operations performed on dynamic routes.

The **Dynamic Route Audit** module tracks every action performed on a dynamic route, recording status, user, and additional details.

This ensures traceability, accountability, and easier debugging of route execution or configuration changes.

Database Tables

Table: dynamic_routes

Stores all dynamic route definitions, including versions, paths, methods, and YAML content.

Column Name	Type	Description
id	BIGINT	Primary key, auto-generated unique ID
routeId	VARCHAR (255)	Unique route identifier from YAML (not nullable)
description	VARCHAR (255)	Description of the route (not nullable)
version	INTEGER	Version number of the route
path	VARCHAR (255)	HTTP path associated with the route (not nullable)
httpMethod	VARCHAR (255)	HTTP method (e.g., GET, POST) for the route
yaml_content	TEXT	YAML source content of the route
active	BOOLEAN	Indicates if this route version is currently active
default_version	BOOLEAN	Marks if this version is the default (not nullable, default false)
created_at	TIMESTAMP	Timestamp when this route was created
created_by	VARCHAR (255)	User who created the route
updated_at	TIMESTAMP	Last updating time
updated_by	VARCHAR (255)	Last updater
comment	VARCHAR (255)	Admin comment for this route version

Table: dynamic_route_audit

Tracks every action performed on dynamic routes, including uploads, updates, deletions, and execution status.

Column Name	Type	Description
id	BIGINT (PK)	Unique identifier for each audit record.
route_id	VARCHAR (255)	Identifier of the dynamic route associated with this audit record.
version	BIGINT	Version number of the dynamic route or configuration at the time of the audit.
action	VARCHAR (255)	Type of action performed on the dynamic route (e.g., UPLOAD , UPDATE , DELETE , START , STOP).
details	TEXT	Additional information or system message related to the action (such as error logs, route configuration details, or operation results).
status	VARCHAR (255)	Status of the route action (e.g., SUCCESS , FAILED , PENDING).
user_email	VARCHAR (255)	Email of the user who performed or triggered the action.
timestamp	TIMESTAMP	Date and time when the action was executed or logged.

Dynamic Routes Process Flow

Create Initial Route

1. An authorized user clicks “**Add New Route**”, opening the create modal.
2. The modal provides an initial YAML template for the route.
3. The user updated Route ID, Route Description, and Route Path.
4. Upon saving, a **new record** is inserted into **dynamic_routes** as the **initial version**.

Update or Manage Route Versions

1. To make changes, the user clicks “Edit” and will be redirected to the API designer.
2. The user modifies the route visually, no manual YAML writing is required.
3. Upon completing edits, the user clicks “**Upload**”.
4. A **new version** is created in **dynamic_routes**, which becomes active while previous versions remain stored for traceability.

Record User Actions in Audit Table

1. Whenever a route is uploaded, updated, started, or stopped, a new record is created in **dynamic_route_audit**.
2. The audit record stores:
 - a. The route ID and version.
 - b. The action type (UPLOAD, UPDATE, DELETE, START, STOP).
 - c. Detailed system messages or errors.
 - d. The user’s email and timestamp.

Route Execution Logging

1. When a route is executed by the middleware, the call is **logged separately** in **middleware_api_call_log**.
2. This ensures separation between **user actions on routes** and **actual runtime API calls**.

Auditing, Maintenance, and Version Control

1. All route versions and audit records are immutable, providing historical traceability.

2. Authorized users can review the audit logs to:
 - a. Verify which routes were executed and by whom.
 - b. Investigate failures or misconfigurations.
 - c. Restore previous versions of a route if necessary.

LLD – Rate Limiting

The Rate Limiting module manages and enforces API request limits across the middleware system.

It supports multiple levels of rate limiting:

1. Global system limits via `rate_limit_config`.
2. Camel integration-level limits via `camel_rate_limit`.
3. Route-specific limits via `custom_camel_rate_limit`, tied to dynamic routes.

This allows fine-grained control over request flows, prevents abuse, and ensures stable performance for all integrated systems and middleware APIs.

Database Tables

Table: `rate_limit_config`

Stores global request limits applicable system-wide.

Column Name	Type	Description
id	BIGINT (PK)	Config ID
limit_requests	BIGINT	Allowed requests in window
window_seconds	BIGINT	Window size in seconds

Table: `camel_rate_limit`

Stores request limits specific to Camel integrations.

Column Name	Type	Description
id	BIGINT (PK)	Config ID
request_limit	INT	Allowed requests in window

Column Name	Type	Description
seconds	INT	Window size in seconds

Table: custom_camel_rate_limit

Stores route-specific request limits for dynamic routes.

Column Name	Type	Description
id	BIGINT (PK)	Config ID
route_id	BIGINT (FK)	Dynamic Route ID
request_limit	BIGINT	Allowed requests in window
seconds	BIGINT	Window size in seconds

Rate Limiting Process Flow

Define Global Rate Limits

1. An authorized user configures **global limits** in the **rate_limit_config** table.
2. These limits **apply to all API requests except those starting with /camel**, which are handled separately.

Define Camel Integration Limits

1. An **authorized user** defines **overall Camel integration limits** in **camel_rate_limit**.
2. These limits govern all **/camel** requests **globally**, ensuring the integration is not overloaded.

Configure Route-Specific Limits

1. An **authorized user** can define **custom limits** for individual dynamic routes under **/camel** in **custom_camel_rate_limit**, linked via **route_id**.
2. This allows individual API endpoints to have stricter or more lenient limits compared to the global or integration-level limits

Enforce Rate Limits at Runtime

1. When a request is processed by the middleware:
 - a. If the path starts with **/camel**:
 - i. Apply the **overall Camel integration limit** from **camel_rate_limit** first.

- ii. Then apply any **custom route limit** from `custom_camel_rate_limit` (if defined).
 - b. If the path is **not /camel**, apply the **global system limit** from `rate_limit_config`.
2. Requests exceeding the limit within the configured time window are **throttled** or **rejected**, ensuring fair usage and preventing overload.

Auditing

1. All actions related to rate limiting configuration are recorded in the **audit_logs** table.
2. This ensures full traceability of all rate-limiting changes, allowing authorized users to review historical configurations, investigate anomalies, and maintain operational compliance.

LLD -Integration Mapping

The Integration Mapping module standardizes how external system data is mapped to middleware APIs, translating external identifiers (external_key and DHIS identifiers like DE_UID or DE_UID.COC_UID) into internal middleware fields. It enables integration with multiple external systems while centralizing mapping rules.

Mappings are persisted in integration_mapping with a unique constraint (dynamic_route_id, integrated_api_id, mapping_type, data, external_key). Configurations are dynamic and can be updated at runtime (API/import) without code changes.

Table: integration_mapping

Column Name	Type	Description
id	BIGINT (PK)	Unique Mapping ID

dynamic_route_id	VARCHAR(100) NOT NULL	Identifier of the dynamic route associated with this mapping. Links a specific route to its corresponding integration.
integrated_api_id	BIGINT (FK)	Reference to the related record in the integrated_apis table representing the external API involved in the mapping.
mapping_type	VARCHAR(50) NOT NULL	Defines the type of mapping
data	TEXT NOT NULL	JSON or structured data defining the mapping details
attribute	VARCHAR(50)	Optional attribute name for additional mapping context or metadata.
external_key	VARCHAR(100) NOT NULL	Unique key representing the external system or resource being mapped.
is_active	BOOLEAN NOT NULL	Indicates whether the mapping is currently active (TRUE) or inactive (FALSE).
notes	TEXT	Optional notes or comments providing extra context about the mapping.
created_at	TIMESTAMP NOT NULL	Date and time when the mapping record was created.
updated_at	TIMESTAMP NOT NULL	Date and time when the mapping record was last updated.
created_by	VARCHAR(255) NOT NULL	Username or process that created the mapping record.

updated_by	VARCHAR(50) NOT NULL	Username or process that last updated the mapping record.
-------------------	----------------------	---

Table: integrated_apis

Column Name	Type	Description
id	BIGINT (PK)	Unique API ID
code	VARCHAR(100) NOT NULL UNIQUE	Unique code used to identify the API within the system.
name	VARCHAR(255) NOT NULL	Descriptive name of the integrated API.
api_url	TEXT NOT NULL	Full endpoint URL of the integrated API.
type	VARCHAR(20) NOT NULL	Type of the API
integrated_system	VARCHAR(100) NOT NULL	code of the external system that this API belongs to
bound_api_code	VARCHAR(100)	route code bound when type=METADATA
use_ou_from_request	BOOLEAN NOT NULL	Indicates whether the API should inherit the Organizational Unit (OU) value from the incoming request.

use_pe_from_request	BOOLEAN NOT NULL	Indicates whether the API should inherit the Processing Entity (PE) value from the incoming request.
is_active	BOOLEAN NOT NULL	Indicates whether the API is active (TRUE) or inactive (FALSE)
description	TEXT	Optional description providing additional context about the integrated API.
created_at	TIMESTAMP NOT NULL	Date and time when the API record was created.
updated_at	TIMESTAMP NOT NULL	Date and time when the API record was last updated.
created_by	VARCHAR(255) NOT NULL	Username or process that created the API record.
updated_by	VARCHAR(50) NOT NULL	Username or process that last updated the API record.

Integration Mapping Process Flow

Ingest / identify context

1. Input: a middleware request routed to a dynamic route . Typical incoming items: route identifier (dynamic_route_id), request params (pe, ou), and the business key the client expects (external identifier). The request may be for analytics or metadata depending on the IntegratedApi configuration.
2. Code entry points: MiddlewareController :
 - a. MiddlewareProcessorService.processMiddlewareRequest(dynamicRouteId, pe, ou, ...).

Lookup active mappings for the route

1. The service loads all active mappings for the route with a single repository call:
 - a. `IntegrationMappingRepository.findActiveRouteMappings(dynamicRouteId)` this returns only mappings where `mapping.isActive = true` AND `mapping.integratedApi.isActive = true` (JOIN FETCH `integratedApi`).

Group mappings by external API

1. Mappings are grouped by their `IntegratedApi` object (`Map<IntegratedApi, List<IntegrationMapping>>`) so the processor can call each external API once and apply many mappings to its response.
 - a. Code `IntegrationMappingService`
`MiddlewareProcessorService.groupMappingsByApi(...)`.

Fetch metadata (org units) per integrated system

1. For each distinct `integratedSystem` (`api.getIntegratedSystem()`), the processor fetches `org-unit` `metadata` using `Dhis2ClientService.fetchOrganisationUnitsMetadata(systemCode, businessApiCode)`.
2. Note: when a metadata API is bound to a dynamic route (`IntegratedApi.type == METADATA` and `boundApiCode == routeId`), `fetchOrganisationUnitsMetadata` finds that metadata API via `IntegratedApiRepository.findActiveMetadataByBoundCode` and uses that API's own `integratedSystem` to fetch metadata.

Execute analytics calls per IntegratedApi

1. For each `IntegratedApi` in the group, the processor calls `Dhis2ClientService.executeApiCall(...)` using that API's `integratedSystem` (not a global `_dhis2Code`). Flags `useOuFromRequest` / `usePeFromRequest` determine whether to take OU/PE from the incoming request or from the configured `apiUrl`.
2. The service validates required params when flags demand them:
 - a. If `api.getUseOuFromRequest()` true → ou must be present.
 - b. If `api.getUsePeFromRequest()` true → pe must be present.

3. Code: `MiddlewareProcessorService.executeDhis2CallsWithDynamicParams(...)`
`Dhis2ClientService.executeApiCall(...)`.

Extract values from analytics responses per mapping

1. For ANALYTICS APIs, the code uses `AnalyticsDataExtractor` (inner class) to parse headers and index values. Extraction rules:
 - a. `mapping.data` holds the DHIS identifier used to find values in analytics rows:
 - i. `DATA_ELEMENT` → `data = "DE_UID"`
 - ii. `DATA_ELEMENT_WITH_DISAGGREGATION` → `data = "DE_UID.COC_UID"`
 - iii. `INDICATOR` → `data = "INDICATOR_UID"`
 - b. For attribute-dimension analytics results, `extractor.getValuesWithAttribute(...)` returns `attribute-id` → value pairs and the code produces one `AttributeDto` per attribute for that `mapping.externalKey`.
 - c. For non-attribute (single value) responses, the code uses `extractor.getDataElementValue(...)` or `getDisaggregatedValue(...)` or `getIndicatorValue(...)`.
2. Matching logic for dx:
 - a. Exact equality of dx header values OR if `mapping.data` contains a dot the extractors handle matching on the DE part (search startsWith when appropriate).
 - b. This matching is implemented in `AnalyticsDataExtractor.matchesDx(...)`.

Build middleware response

1. The processor accumulates values into aggregation rows keyed by (ou, period) and appends attributes per `mapping.externalKey`, producing the final `MiddlewareResponseDto` with `MiddlewareRowDto` entries.
2. Code: `buildMiddlewareResponse(...)` and `processMapping(...)`.

Validation & mapping rules enforcement

1. The entity enforces mapping rules at persist/update time (`IntegrationMapping.validateData()`):

- a. `DATA_ELEMENT_WITH_DISAGGREGATION` must contain exactly one dot and two parts.
 - b. `DATA_ELEMENT` must not contain a dot.
 - c. `INDICATOR` must satisfy minimal length rules.
2. The service `validateMappingForImport(...)` and `validateMapping(...)` mirror these checks for imports and admin calls.

Error handling & fallback

1. If no active mappings returned for a route, the processor logs a warning and returns an empty response (`MiddlewareResponseDto` with empty rows).
 - a. Code: `processorService` returns empty rows when `mappings.isEmpty()`.
2. If an API call fails, the processor logs the error and continues; `responses.put(api, new HashMap<>())` so other APIs still contribute.
3. If a mapping validation fails during import/create, the import returns errors and optionally updates/ignores depending on `updateExisting` flag.

Admin / runtime updates

1. Mappings can be created/updated via `IntegrationMappingService` and Admin controller endpoints, or imported from Excel; changes apply at runtime (no code change required).
2. Unique constraint enforced in DB and entity: (`dynamic_route_id`, `integrated_api_id`, `mapping_type`, `data`, `external_key`) prevents duplicates.

Auditing

1. All actions related to rate limiting configuration are recorded in the **audit_logs** table.
2. This ensures full traceability of all rate-limiting changes, allowing authorized users to review historical configurations, investigate anomalies, and maintain operational compliance.

Transaction logging

Every runtime call that uses a mapping (any request routed to a mapped dynamic route or any mapping-related API call) is recorded as a transaction log on ingress and updated on completion. These logs support debugging, retry correlation and performance monitoring.

LLD -User Management

The **Users Management** module provides authentication, authorization, and access control across the middleware system. It defines users, their roles, and the permissions associated with those roles. This ensures secure access, fine-grained privilege control, and a scalable way to manage system and business users.

Users are linked to roles, and roles are associated with one of permissions (functional capabilities) or route-based access (API-level restrictions), this depends on the Role Type. All relationships are stored in normalized relational tables, enabling dynamic updates without modifying application logic.

Database Tables

Table: users

Defines users.

Column Name	Type	Description
id	BIGINT (PK)	User ID
user_name	VARCHAR(255) NOT NULL	Unique username of the user
email	VARCHAR(255) UNIQUE NOT NULL	Unique user email (used for login/notifications)
status	VARCHAR(255)	User status (e.g., ACTIVE, INACTIVE)
created_at	TIMESTAMP NOT NULL	Timestamp when user was created
created_by	VARCHAR(255) NOT NULL	Username or system process that created the user record.
updated_at	TIMESTAMP NOT NULL	Timestamp of last update
updated_by	VARCHAR(255) NOT NULL	Username or system process that last updated the user record.
password	VARCHAR(255) NOT NULL	Encrypted user password

Table: role

Defines Roles.

Column Name	Type	Description
id	BIGINT (PK)	Role ID
role_name	VARCHAR(255) UNIQUE NOT NULL	Unique name of the role
role_type	VARCHAR(255)	Role type (e.g., USER, SYSTEM_USER)
created_at	TIMESTAMP NOT NULL	Timestamp when role was created
created_by	VARCHAR(255) NOT NULL	Username or system process that created the role record.
updated_at	TIMESTAMP NOT NULL	Timestamp of last update
updated_by	VARCHAR(255) NOT NULL	Username or system process that last updated the role record.

Table: permissions

Defines System Permissions.

Column Name	Type	Description
id	BIGINT (PK)	Permission ID

name	VARCHAR(255) UNIQUE NOT NULL	Unique permission identifier (e.g., CREATE_USER, VIEW_LOGS)
-------------	------------------------------------	---

Table: routes_permissions

Defines Routes Permissions.

Column Name	Type	Description
id	BIGINT (PK)	Route Permission ID
route_id	VARCHAR(255)	Identifier of the API route controlled by permission

Table: users_roles (Join Table)

Defines the mapping between Users and Roles.

Column Name	Type	Description
user_id	BIGINT (FK → users.id)	Associated user
role_id	BIGINT (FK → role.id)	Assigned role

Table: role_permissions (Join Table)

Defines the mapping between Roles and Permissions “Just for **Real Users**”.

Column Name	Type	Description
role_id	BIGINT (FK → role.id)	Associated role

permission_id	BIGINT (FK → permissions.id)	Permission granted to role
----------------------	------------------------------	----------------------------

Table: role_routes_permissions (Join Table)

Defines the mapping between Roles and Route Permissions “Just for **System Users**”.

Column Name	Type	Description
role_id	BIGINT (FK → role.id)	Associated role
route_id	BIGINT (FK → routes_permissions.id)	Route permission granted

Table: user_tokens

Defines the generated tokens.

Column Name	Type	Description
id	BIGINT	Primary key, auto-generated unique identifier for the token.
user_id	BIGINT (FK)	Foreign key referencing users.id. Associate the token with a specific user.
token	VARCHAR(10000)	The token string (e.g., JWT or session token). Maximum length 10,000 characters.
is_valid	BOOLEAN	Flag indicating if the token is valid (true) or invalid/revoked (false).
created_at	TIMESTAMP	Timestamp of when the token was created.
expires_at	TIMESTAMP	Timestamp of when the token expires.

Users & Roles Management Process Flow

User Creation & Role Assignment

1. An authorized user creates a user with username, email, and password.
2. Users are assigned one or more roles via users_roles.
3. Role type can be either user or system user role type.

Role Definition & Permission Mapping

1. Roles are created in the role table.
2. Permissions (functional access) are linked via role_permissions.
3. Route-based restrictions (API-level access) are defined via role_routes_permissions.

Authentication

1. During login, user credentials are validated against the users table.
2. Status (ACTIVE/INACTIVE) determines if login is permitted.

Authorization

1. Middleware fetches the user's roles.
2. Based on roles, the system collects:
 - a. Functional permissions from permissions
 - b. Route/API-level permissions from routes_permissions
3. Authorization is enforced before processing any request.

Auditing & Dynamic Updates

1. **All actions** performed on users, roles, permissions, and role mappings are recorded in the **audit_logs** table.
2. Auditing ensures full traceability of user and role management activities and supports compliance and troubleshooting.
3. All changes are **applied dynamically** without restarting the middleware.

LLD -Config Settings and Integrated Systems

The **Config Settings and Integrated Systems** module manages **dynamic system parameters** and **external system integrations**.

It allows authorized users to define, update, and maintain system-wide configuration keys and store external system information for seamless middleware integration.

Database Tables

Table: config

Defines system-wide configuration parameters that can be dynamically updated.

Column Name	Type	Description
id	UUID	Unique identifier for each configuration entry.
key	VARCHAR(255) UNIQUE NOT NULL	Unique key that identifies the configuration parameter (e.g., EMAIL_SERVER_HOST , MAX_LOGIN_ATTEMPTS).
value	TEXT	The stored value associated with the configuration key (can be text, JSON, number, etc.).
type	VARCHAR(255)	The data type of the configuration (e.g., TEXT , NUMBER).

description	VARCHAR(255)	A brief explanation of what the configuration key controls or represents.
created_at	TIMESTAMP	Date and time when the configuration entry was created.
created_by	VARCHAR(255)	Username or process that created the configuration record.
updated_at	TIMESTAMP	Date and time when the configuration entry was last updated.
updated_by	VARCHAR(255)	Username or process that last updated the configuration record.

Table: integrated_system

Stores information about external systems integrated with the middleware.

Column Name	Type	Description
id	BIGINT (PK)	Unique identifier for each integrated system record.
code	VARCHAR (255) UNIQUE NOT NULL	Unique code or identifier representing the integrated system within the application.
protocol	VARCHAR(255)	Communication protocol used (e.g., HTTP , HTTPS , FTP , SMTP).

host	VARCHAR(255)	Hostname or IP address of the integrated system's server.
port	VARCHAR (255)	Network port number used to connect to the integrated system.
authentication_type	VARCHAR (255)	The authentication method used for connecting to the integrated system (e.g., BASIC , TOKEN , OAuth2).
username	VARCHAR (255)	Username used for authentication with the integrated system.
password	VARCHAR (255)	Password used for authentication with the external system (stored securely or encrypted).
token	VARCHAR (1000)	Authentication token for systems using token-based access.
description	VARCHAR (255)	Brief description of the integrated system and its purpose.
additional_key1	VARCHAR(255)	Optional key name for additional system configuration or metadata.
additional_value1	VARCHAR(255)	Value corresponding to additional_key1 . Used for custom or extra integration parameters.
additional_key2	VARCHAR(255)	Optional second key name for extended configuration or metadata.
additional_value2	VARCHAR(255)	Value corresponding to additional_key2 . Used for custom or extra integration parameters.
created_at	TIMESTAMP	Date and time when the record was created.

created_by	VARCHAR (255)	Username or system process that created the record.
updated_at	TIMESTAMP	Date and time when the record was last updated.
updated_by	VARCHAR	Username or system process that last updated the record.

Config and Integration Management Process Flow

Manage Configuration Parameters

1. An **authorized user** creates or updates a configuration parameter in **config**.
2. Each configuration entry includes a unique key, value, type, and description.
3. Changes are **dynamic**, the middleware applies updates **without restarting**.

Add or Update Integrated Systems

1. An **authorized user** adds or updates external systems in **integrated_system**.
2. Each entry contains Connection details, Authentication, and optional Metadata
3. This allows the middleware to access external systems securely and dynamically.

Apply and Use Configurations

1. Integrated system details are used to establish connections and communicate with external systems.
2. Any updates made by authorized users are reflected immediately across the system without downtime.

Auditing and Traceability

1. **All user actions** on configuration and integrated system entries are stored in **audit_logs**.

Swagger API Documentation Integration

To ensure the designed APIs are well-documented and easily consumable by developers and integrators, **Swagger (OpenAPI)** will be integrated into the middleware system. This integration provides automatic generation of API documentation based on the Spring Boot controllers.

Key Design Points:

1. Automatic Documentation

- a. Annotated Spring controllers will automatically appear in the Swagger UI.
- b. The documentation includes details such as endpoints, HTTP methods, input/output schemas, query/path parameters, and response codes.

2. Swagger UI Access

- a. Once enabled, the Swagger UI will be accessible via a standard endpoint (e.g., /swagger-ui.html or /api-docs).
- b. API specifications will also be available in JSON/YAML format (e.g., /v3/api-docs) for integration with external tools.

3. Benefits

- a. Developers and testers can interactively explore and test APIs directly from the Swagger UI.
- b. Eliminates manual API documentation effort by keeping documentation in sync with implementation.
- c. Facilitates onboarding of new developers and external system integrators by providing clear, always-up-to-date reference material.

Camel Route Designer

With an emphasis on Spring Boot support, enrichment features (such as testing, deploying, route listing, versioning, and simulation), and usability, below is a comparison of the best Apache Camel designers will assist in selecting one for end-user exposure.

Designer's feature comparisons

Let's first list each designer's features so we can finally compare them to conclude the best fit for our use case.

1. Kaoto

Type: Low-/No-code visual designer (Web app + VS Code)

- Spring Boot Support: Full YAML & XML DSL generation that plugs directly into Camel Spring Boot projects
camel.apache.org+[4camel.apache.org](https://camel.apache.org)+[4christophd.github.io](https://christophd.github.io)+[4apache.googlesource.com](https://apache.googlesource.com)+[5developers.redhat.com](https://developers.redhat.com)+[5medium.com](https://medium.com)+[5](#).
- Testing/Simulation: In VS Code and Kaoto, you can run routes locally via Camel JBang start/pause/stop route lifecycles without leaving the UI
developers.redhat.com+[5developers.redhat.com](https://developers.redhat.com)+[5d2bhdhkti9t3uj.cloudfront.net](https://d2bhdhkti9t3uj.cloudfront.net)+[5](#).
- Enrichment Features:
 - Live deploy and runtime control (Run/Pause/Stop buttons) .
 - Documentation generation from routes developers.redhat.com.
 - Data Mapper and Form engine for transforming data visually
developers.redhat.com.
- Ease of Use: Highly intuitive drag-and-drop with code sync, multi-DSL support including YAML and XML developers.redhat.com+[3developers.redhat.com](https://developers.redhat.com)+[3medium.com](https://medium.com)+[3](#).
- Cost: Free, open-source (Apache 2.0) github.com.
- Best For Teams building Spring Boot and Camel flows seeking visual design, iteration, and testing in one tool.

2. Camel Karavan

Type: Low-code visual designer (Web + VS Code extension) aimed at Camel K

- Spring Boot Support: Can export to Spring Boot, but route execution integrated via Karavan is optimized for Camel K/K-native
github.com+[9camel.apache.org](https://camel.apache.org)+[9medium.com](https://medium.com)+[9](#).
- Testing/Simulation: Supports local dev-run via Camel JBang in VS Code, deploy pipelines trigger builds and live logs
christophd.github.io+[2camel.apache.org](https://camel.apache.org)+[2d2bhdhkti9t3uj.cloudfront.net](https://d2bhdhkti9t3uj.cloudfront.net)+[2](#).
- Enrichment Features:

- OpenAPI ⇔ REST DSL generation; built-in support for Kamelets and complex EIPs slides.delawen.com+6camel.apache.org+6github.com+6.
- GitOps via projects stored in Git and Tekton pipelines for deploying to Kubernetes camel.apache.org+1apache.google.com+1.
- Runtime live logs and tracing in web view awesome.ecosyste.ms+1apache.google.com+1.
- Ease of Use: Visual, pattern-rich, ideal for cloud-integration developers familiar with Kamelets.
- Cost: Free, open-source (Apache 2.0).
- Best For Users building in Kubernetes environments with Camel K, collaborative GitOps approach.

3. Camel Designer (VS Code Plugin)

Type: IDE-integrated graphical route builder

- Spring Boot Support: Supports Java, XML, and YAML DSL, code sync builds Spring Boot compatible route classes d2bhdhkti9t3uj.cloudfront.net+5camel.apache.org+5github.com+5camel.apache.org+4developers.redhat.com+4medium.com+4.
- Testing/Simulation: Leverages VS Code Camel tooling, debugging and running routes possible via built-in camel support camel.apache.org+4developers.redhat.com+4d2bhdhkti9t3uj.cloudfront.net+4.
- Enrichment Features:
 - Graphical representation in the editor.
 - Code completion, validation, and debugging.
- Ease of Use: Good for developers wanting visual cues alongside code, less full-featured UI than Kaoto.
- Cost: Free, open-source.
- Best For Developers who prefer working within VS Code with occasional graphical route design needs.

4. Fuse Tools (JBoss Tools – Eclipse)

Type: Visual route editor for Eclipse

- Spring Boot Support: Generates XML DSL, fully integrates with Camel Spring Boot projects [github.com+4d2bhdhkti9t3uj.cloudfront.net+4](https://github.com/4d2bhdhkti9t3uj.cloudfront.net).
- Testing/Simulation: Supports debugging and in-Eclipse runtime control.
- Enrichment Features:
 - Mature EIP support via palette.
 - Integrated route testing, debugging, and profiling.
- Ease of Use: Rich feature set, but Eclipse dependency adds weight.
- Cost: Free, open-source under Eclipse umbrella.
- Best For XML-heavy teams already working in Eclipse.

5. Hawtio

Type: Runtime web console

- Spring Boot Support: Works with deployed Camel apps, including Spring Boot.
- Testing/Simulation: No design features, only monitors deployed routes.
- Enrichment Features:
 - Endpoint browser, metrics, tracing, runtime graphs.
- Ease of Use: Great for production monitoring, no design capability.
- Cost: Free, open-source.
- Best For Operational teams needing runtime observability.

6. Syndesis / Fuse Online

Type: Enterprise iPaaS

- Spring Boot Support: Generates routes deployable to Camel runtimes including Spring Boot.
- Testing/Simulation: Built-in validation and runtime testing.
- Enrichment Features:
 - API management, connectors, dashboards, RBAC.
- Ease of Use: Enterprise-ready but heavy, often requires OpenShift.
- Cost: Free OSS with commercial options in Red Hat subscriptions.
- Best For Enterprise environments needing full iPaaS capabilities.

Comparison Table

Feature	Kaoto	Karavan	Camel Designer (VS Code)	Fuse Tools (Eclipse)	Hawtio	Syndesis / Fuse Online
Visual Design	Drag & drop and forms	Drag & drop and REST designer	Graphical with code sync	Eclipse palette and visual	Only runtime	Drag & drop and API tools
DSL Support	YAML, XML	YAML (Camel K) and OpenAPI import	Java, XML, YAML	XML	Runtime only	YAML/XML
Spring Boot Integration	Full support	Partial (export only)	Full support	Full support	Monitor only	Full support
Local Testing/Simulation	JBang run/pause/stop	JBang & dev mode pipelines	VS Code debugging/run	Debug & run via Eclipse	No design/test features	Validation/testing
GitOps / Versioning	Limited (file-based)	Git+Tekton pipelines	Code-based via Git	Project files in workspace	Monitoring only	Projects and CI integration
Route Deployment	Manual export/API integration	Kubernetes & Spring Boot	Manual build/run	Manual config	Not applicable	Built-in deployment tools
Runtime Monitoring	No	Logs+tracing	VS Code tooling	Eclipse runtime view	Metrics and tracing	Dashboards and alerts
Ease of Use	5	4	4	4	3	3
Lightweight vs Heavy	Lightweight	Medium	Lightweight	Heavy (Eclipse)	Lightweight	Heavy (iPaaS)

Conclusion

Since our objective is to enable end-users to design, test, deploy, and manage Camel routes in a Spring Boot context beside Kaoto stands out for its user experience, tooling integration, and light footprint then Kaoto is recommended.

Kaoto Web App and Kaoto VS Code Plugin

Kaoto Web App

- A browser-based Camel route designer
- Runs as a standalone web service (via Docker, Quarkus JAR, or embedded in another app)
- Visual designer that supports drag-and-drop + YAML editor sync
- Uses PatternFly React frontend, you can modify the web app UI/UX

Capabilities:

- Visual route design (REST, EIPs, processors, Kamelets, etc.)
- YAML code view + sync
- Form-based editing of properties
- Can run in a shared environment (multi-user capable via custom auth)
- Easily integrate custom actions:
- Add buttons (e.g. *Deploy Route*, *Validate*, *Test Route*)
- Call internal APIs
- Add custom panels (e.g. route list, deploy status)

Customization potential:

- High, full access to React source code → fork and modify, add API calls, custom deploy buttons, enrich UI
- Possible to package as Electron app for desktop if needed

Deployment:

- Any HTTP server
- Expose securely with reverse proxy, auth, etc.

VS Code Plugin

- A Visual Studio Code extension that adds a Kaoto-powered visual editor inside VS Code
- Syncs with your code files (YAML/XML/Java DSL)

Capabilities:

- Visual design within VS Code
- Code and visual views are always in sync
- Can work offline in local VS Code

- Can run/test locally using Camel JBang (with VS Code terminal or tasks)

Customization potential:

Low:

- You cannot easily change or extend the plugin's functionality without modifying and building the extension yourself.
- VS Code plugins have stricter APIs for customization
- Users must run VS Code (developer environment)

Deployment:

- Installed individually by each user via VS Code Marketplace
- No shared or central environment → each user works locally

Comparison: Kaoto Web App vs Kaoto VS Code Plugin

Feature / Criteria	Kaoto Web App	Kaoto VS Code Plugin
Target User	End users, business users, devs (via web)	Developers using VS Code locally
Deployment Model	Central server, shared, browser-based	Per-user install in VS Code IDE
Customization	Full React code, easy to modify	Hard, requires extension rebuild
Add Deploy/Test Buttons	Easy: add React components and API calls	Not practical without rebuilding ext
Multi-user Support	Yes (via web, with auth layers)	No, local only
Offline use	Needs server access	Yes, local VS Code project
Runs in Dev IDE	(browser separate from code IDE)	Yes, part of VS Code workflow
Visual Editing	Drag/drop + form editing and YAML sync	Same, inside VS Code
Deployability	Can build deploy panel / API calls	No direct deploy unless scripted
Ease of exposing to users	(5) Web URL, easy to share	(1) Each user must install plugin

Summary

Kaoto Web App is your best choice if:

- You want to expose a designer to end users (not just devs)
- You want to add deploy, test, list routes features easily
- You want central hosting + multi-user access
- You want to customize and enrich the UI

Kaoto VS Code Plugin is good if:

- Your users are developers working inside VS Code
- You don't need centralized deploy/test features in the designer itself
- You want offline / local route design

Final recommendation

For our case (end-user design + deploy + enrichment) along with Kaoto Web App + custom backend enhancements is the best choice

Integration Project Phases

The integration project is structured across seven distinct phases. Commencing with project initiation and requirement gathering, it progresses through meticulous planning and design. Subsequently, the integration is executed and rigorously tested to ensure adherence to specifications. Upon successful validation, the system is deployed, followed by ongoing support and maintenance. The project concludes with a comprehensive review to evaluate its effectiveness and identify areas for improvement, establishing a solid foundation for future endeavors.

Table 4: Process for integration

Phase	Objective	Key Activities
Phase 1: Initiation	Establish a clear understanding of client expectations and gather all necessary information.	● Conduct kickoff meetings to discuss the scope and objectives. ● Review completed surveys to ascertain requirements.
Phase 2: Planning and Design	Develop a detailed integration plan that aligns with client needs.	● Design system architecture and data flow. ● Resource allocation and scheduling. ● Establish project management practices.
Phase 3: Execution	Implement the integration according to the project plan.	● Develop integration solutions. ● Regular project updates and stakeholder meetings. ● Continuous testing during development.
Phase 4: Testing and Validation	Ensure the integration meets all technical and business requirements.	● Conduct comprehensive testing. ● User acceptance testing. ● Troubleshooting and adjustments.
Phase 5: Deployment	Go-live with the integrated system.	● Finalize deployment strategy. ● Monitor and address any issues during the initial live period.
Phase 6: Support and Maintenance	Provide ongoing support and updates.	● Deliver training and documentation. ● Establish a support desk. ● Regular performance reviews and system updates.
Phase 7: Review	Assess the success of the integration project and identify areas for improvement.	● Conduct a post-implementation review. ● Gather feedback from all stakeholders. ● Document lessons learned and apply these to future projects.